

a) Diseño de Tipos Abstractos de Datos (TAD)

La jerarquía de vehículos se mantiene intacta, pero cambia la forma en que la clase gestora los agrupa.

TAD Empresa: Representa el sistema que administra la colección de vehículos mediante una secuencia lineal.

TAD Vehículo (Clase Abstracta): Generalización de los vehículos, centralizando los atributos comunes.

TAD Auto (Subclase de Vehículo): Especialización para automóviles.

TAD Moto (Subclase de Vehículo): Especialización para motocicletas.

Observaciones y justificación del diseño:

1. **Estructura Secuencial:** Se opta por una colección de tipo Lista (List o Arreglo dinámico) para el TAD Empresa. A diferencia del Diccionario, la Lista mantiene el orden de inserción de los vehículos.
2. **Impacto en el Rendimiento:** Al usar una secuencia, las operaciones que requieren buscar un vehículo específico por su matrícula (como las altas para evitar duplicados, y las bajas) pasan a tener una complejidad de $O(N)$, ya que el sistema debe recorrer la lista elemento por elemento en el peor de los casos.
3. **Herencia y Polimorfismo:** Se mantiene la promoción del atributo precioBase a la superclase y el uso del método polimórfico precioAlquilerDiario() para garantizar la extensibilidad del código.

b) Desglose de Requerimientos

Requerimiento 1a: Alta de vehículo.

Entrada: Matrícula, Marca, Año, Precio Base, Tipo de Vehículo (Auto/Moto), y el atributo específico (Cantidad de Puertas o Cilindrada).

Salida: Confirmación de éxito o mensaje de error.

Proceso:

1. Recorrer la lista secuencial de vehículos desde el principio hasta el final.
2. Comparar la matrícula recibida con la matrícula de cada vehículo en la lista.
3. Si se encuentra una coincidencia, abortar y retornar error (matrícula duplicada).
4. Si se llega al final de la lista sin encontrar coincidencias, instanciar un objeto Auto o Moto con los datos de entrada.
5. Agregar el nuevo objeto al final de la lista secuencial.
6. Retornar confirmación de éxito.

Requerimiento 1b: Baja de vehículo.

Entrada: Matrícula del vehículo a dar de baja.

Salida: Confirmación de éxito o mensaje de error.

Proceso:

1. Recorrer la lista secuencial de vehículos.
2. Comparar la matrícula de entrada con la de cada vehículo iterado.
3. Si se encuentra una coincidencia, eliminar el objeto de esa posición en la lista.
4. Retornar confirmación de éxito.
5. Si se recorre toda la lista sin encontrar la matrícula, retornar error (vehículo inexistente).

Requerimiento 2: Calcular suma de precios de alquiler diario.

Entrada: Ninguna.

Salida: Valor numérico real (Monto total).

Proceso:

1. Inicializar sumaTotal = 0.
2. Iterar secuencialmente sobre la lista de vehículos.
3. Invocar el método polimórfico precioAlquilerDiario() en cada objeto.
4. Acumular el resultado en sumaTotal.
5. Finalizada la iteración, retornar sumaTotal.

Requerimiento 3: Obtener vehículos con mayor y menor precio de alquiler diario.

Entrada: Ninguna.

Salida: Dos entidades (Vehículo mayor precio, Vehículo menor precio).

Proceso:

1. Validar que la lista no esté vacía. Si lo está, retornar error.
2. Asignar el primer vehículo de la lista a las variables vehiculoMayor y vehiculoMenor.
3. Calcular su precio y asignarlo a maxPrecio y minPrecio.
4. Iterar sobre el resto de los elementos de la lista.
5. Calcular precioActual = precioAlquilerDiario() para el elemento en curso.
6. Actualizar maxPrecio y vehiculoMayor si precioActual > maxPrecio.
7. Actualizar minPrecio y vehiculoMenor si precioActual < minPrecio.

8. Al terminar, retornar ambos vehículos.

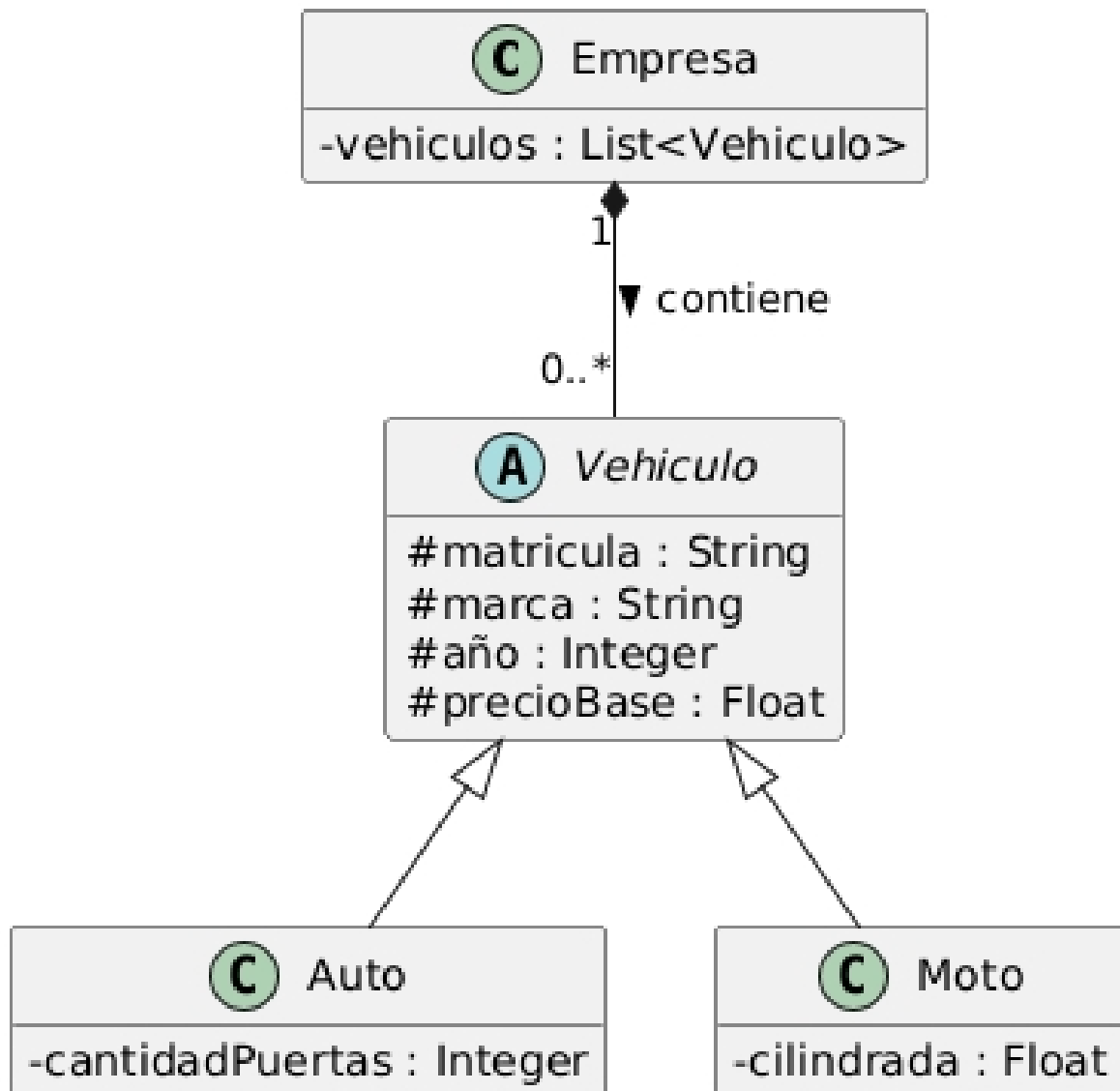
Requerimiento 4: Contar motos por cilindrada.

Entrada: Cilindrada a buscar.

Salida: Cantidad de motos coincidentes (Valor entero).

Proceso:

1. Inicializar contador = 0.
2. Iterar secuencialmente la lista de vehículos.
3. Comprobar si el elemento actual es instancia de Moto.
4. Si lo es, realizar un casteo a Moto.
5. Obtener su cilindrada mediante el selector correspondiente.
6. Si coincide con la entrada, incrementar contador.
7. Retornar contador al finalizar.



Especificación detallada de cada clase

1. Clase: Empresa

Atributos:

vehiculos: Secuencia lineal (Lista) que almacena instancias de Vehiculo.
Visibilidad privada.

Constructor:

Empresa(): Inicializa la lista vehiculos vacía.

Selectores/Modificadores: No se exponen getters/setters directos de la lista.

Métodos específicos:

altaVehiculo(...)

bajaVehiculo(String matricula)
calcularSumaPreciosDiarios() -> Float
obtenerExtremosPrecioDiario() -> Tuple/Array
contarMotosPorCilindrada(Float cilindrada) -> Integer

2. Clase: Vehiculo (Abstracta)

Atributos:

matricula (String): Protegido.
marca (String): Protegido.
año (Integer): Protegido.
precioBase (Float): Protegido.

Constructor:

Vehiculo(String mat, String mar, Integer a, Float pBase)

Selectores: getMatricula(), getMarca(), getAño(), getPrecioBase().

Modificadores: setMarca(), setAño(), setPrecioBase().

Métodos polimórficos:

abstract Float precioAlquilerDiario()

3. Clase: Auto (Hereda de Vehiculo)

Atributos:

cantidadPuertas (Integer): Privado.

Constructor:

Auto(String mat, String mar, Integer a, Float pBase, Integer cantPuertas)

Selectores/Modificadores: getCantidadPuertas(), setCantidadPuertas().

Métodos polimórficos:

Float precioAlquilerDiario(): Retorna getPrecioBase() + (cantidadPuertas * 100).

4. Clase: Moto (Hereda de Vehiculo)

Atributos:

cilindrada (Float): Privado.

Constructor:

Moto(String mat, String mar, Integer a, Float pBase, Float cil)

Selectores/Modificadores: getCilindrada(), setCilindrada().

Métodos polimórficos:

Float precioAlquilerDiario(): Retorna $\text{getPrecioBase()} + (\text{cilindrada} * 0.5)$.